

Postman API Testing Report

DummyJSON

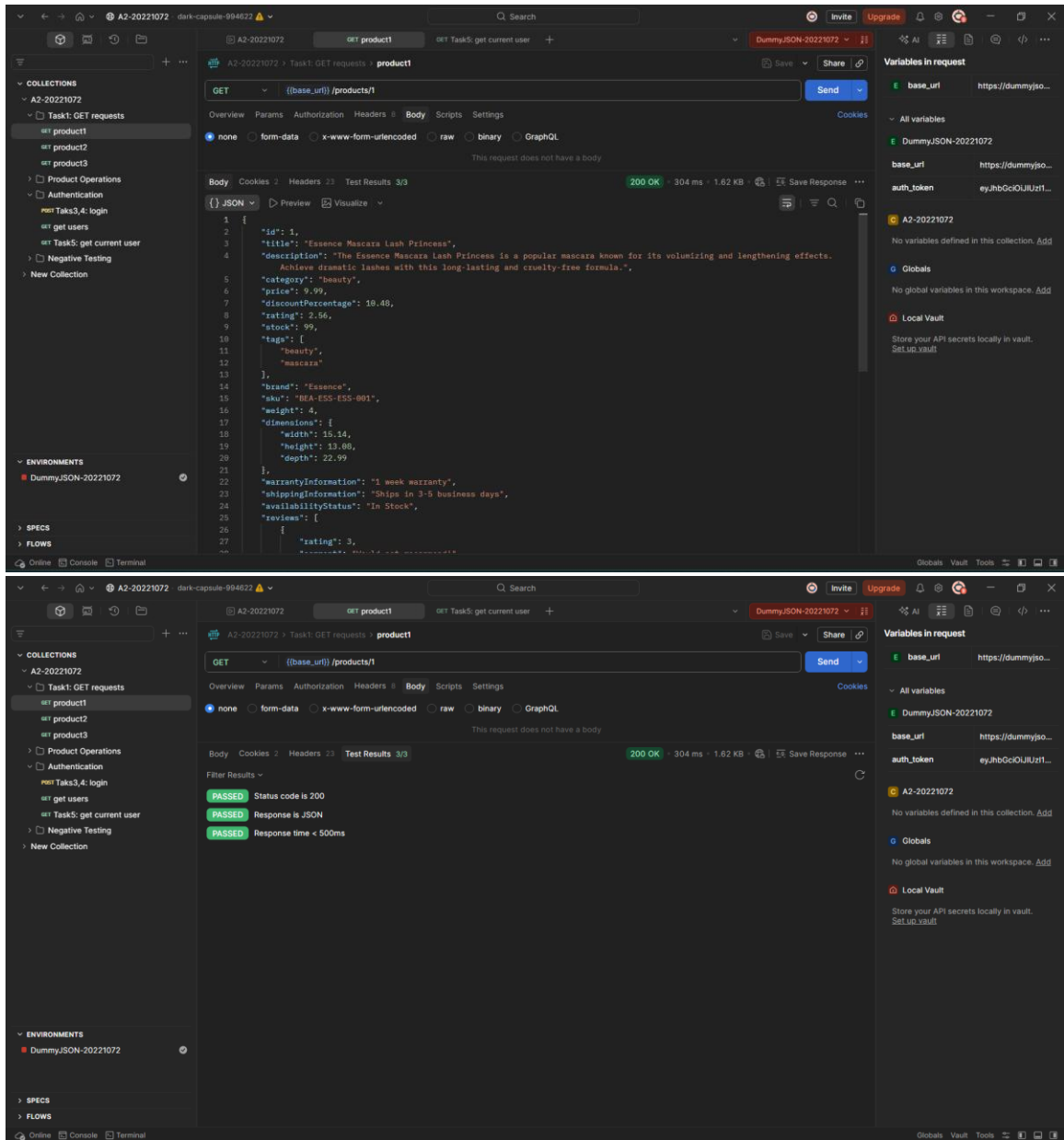
Course: Software Testing

TA: Mahmoud Hadad

Team Members:

Salman Ahmed – 20221072,
Abdelrahman Mohammed – 20220195,
Khaled Mostafa - 20220120

1. GET Product Request



Figure

1: GET Product request with validation tests (Status 200, JSON response, response time)

2. POST Add Product

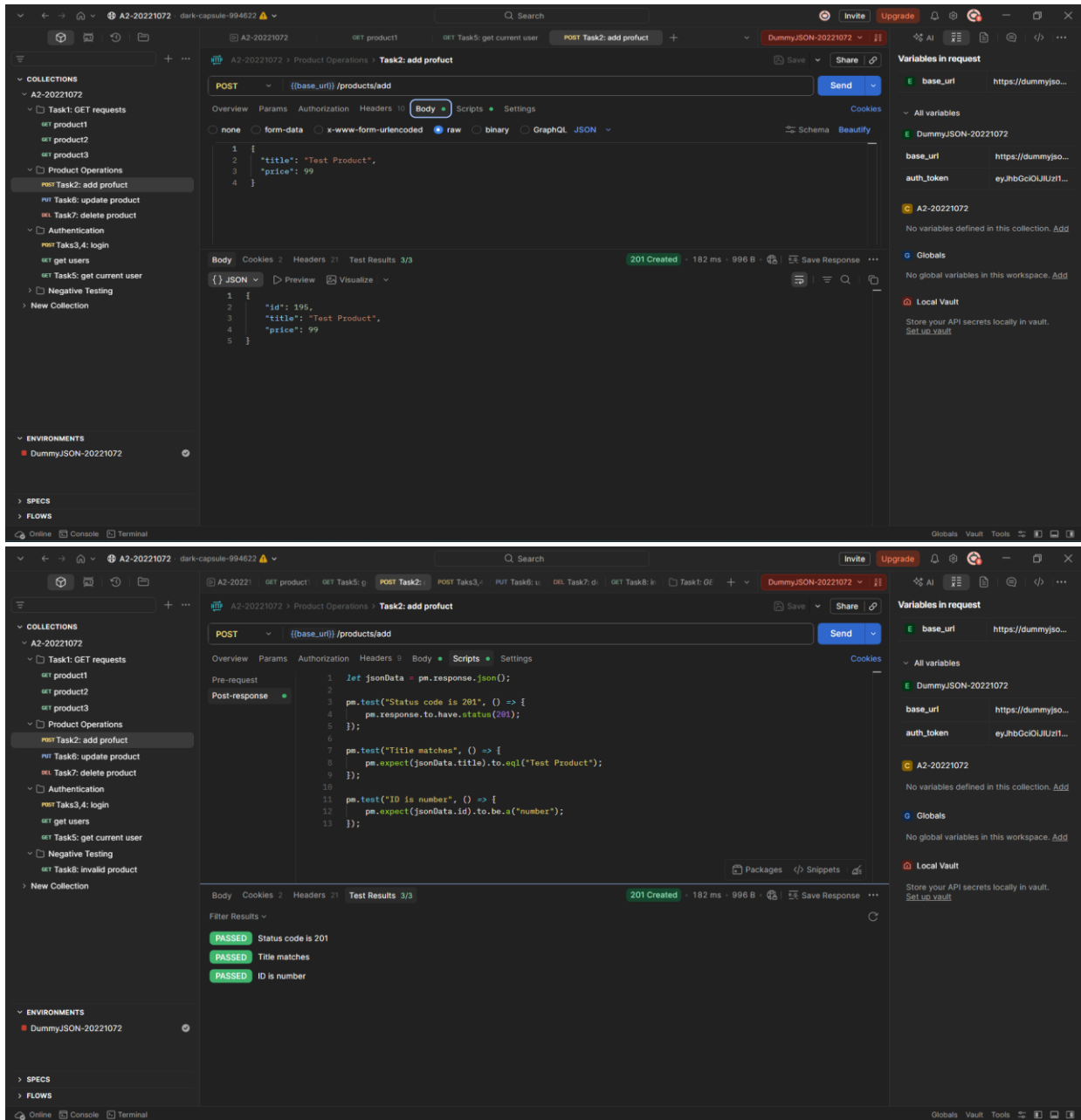
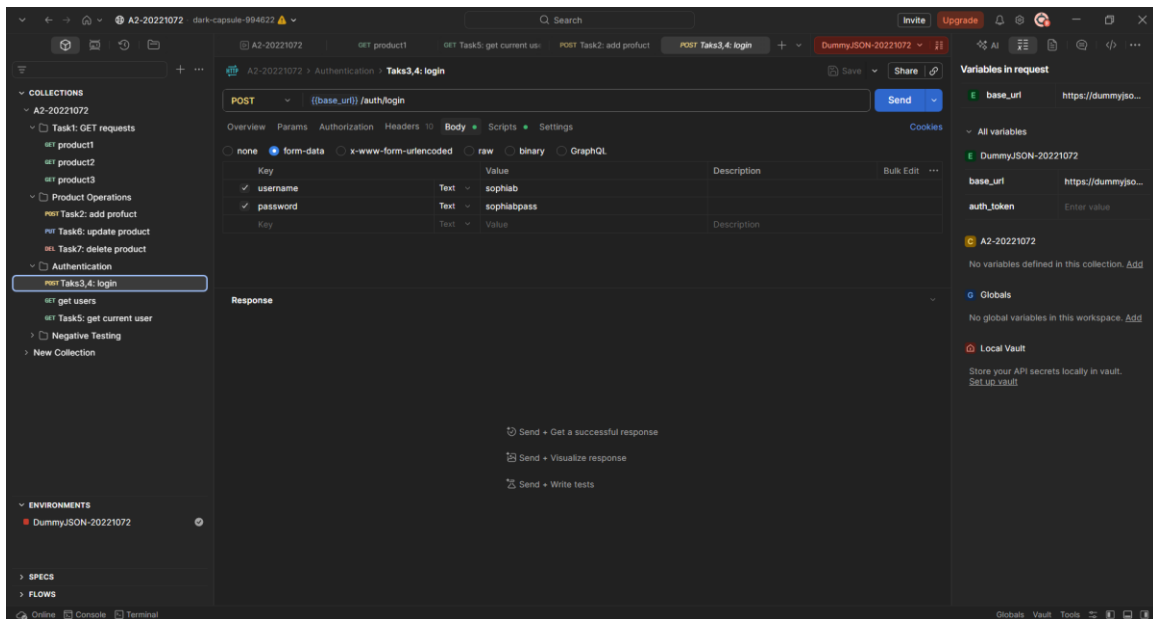


Figure 2: POST Add Product with validation of title and ID

3. POST Login

Before Requesting



After requesting

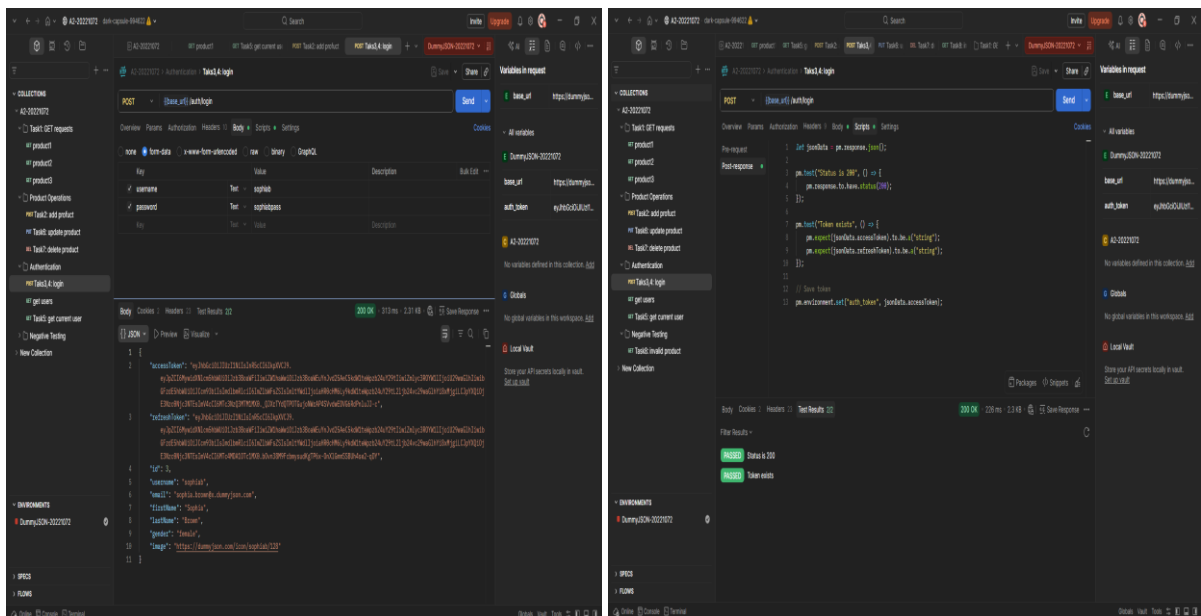


Figure 3: Login request validating token generation and saving to environment

4. Environment Variables

Before

Variables in request

E base_url	https://dummyjso...
-------------------	---------------------

▼ All variables

E DummyJSON-20221072

base_url	https://dummyjso...
auth_token	Enter value

C A2-20221072

No variables defined in this collection. [Add](#)

G Globals

No global variables in this workspace. [Add](#)

Local Vault

Store your API secrets locally in vault.
[Set up vault](#)

After

Variables in request

E base_url	https://dummyjso...
-------------------	---------------------

▼ All variables

E DummyJSON-20221072

base_url	https://dummyjso...
auth_token	eyJhbGciOiJIUzI1...

C A2-20221072

No variables defined in this collection. [Add](#)

G Globals

No global variables in this workspace. [Add](#)

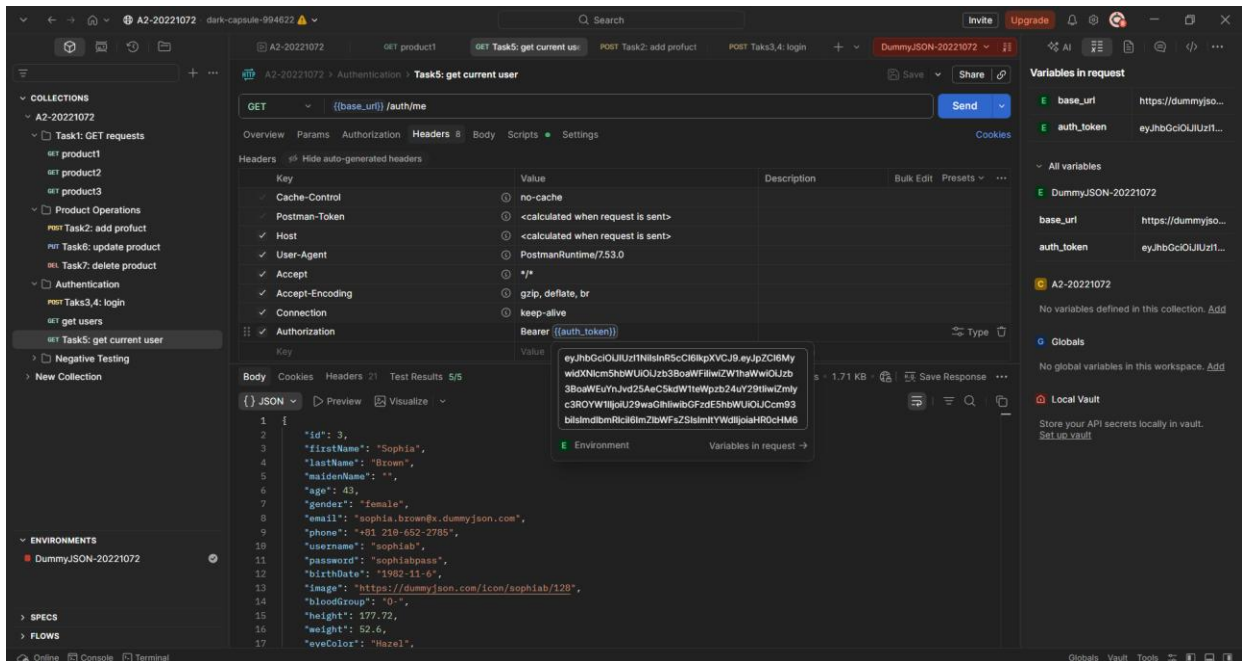
Local Vault

Store your API secrets locally in vault.
[Set up vault](#)

Figure 4: Environment variables showing dynamically stored authentication token

5. GET Current User

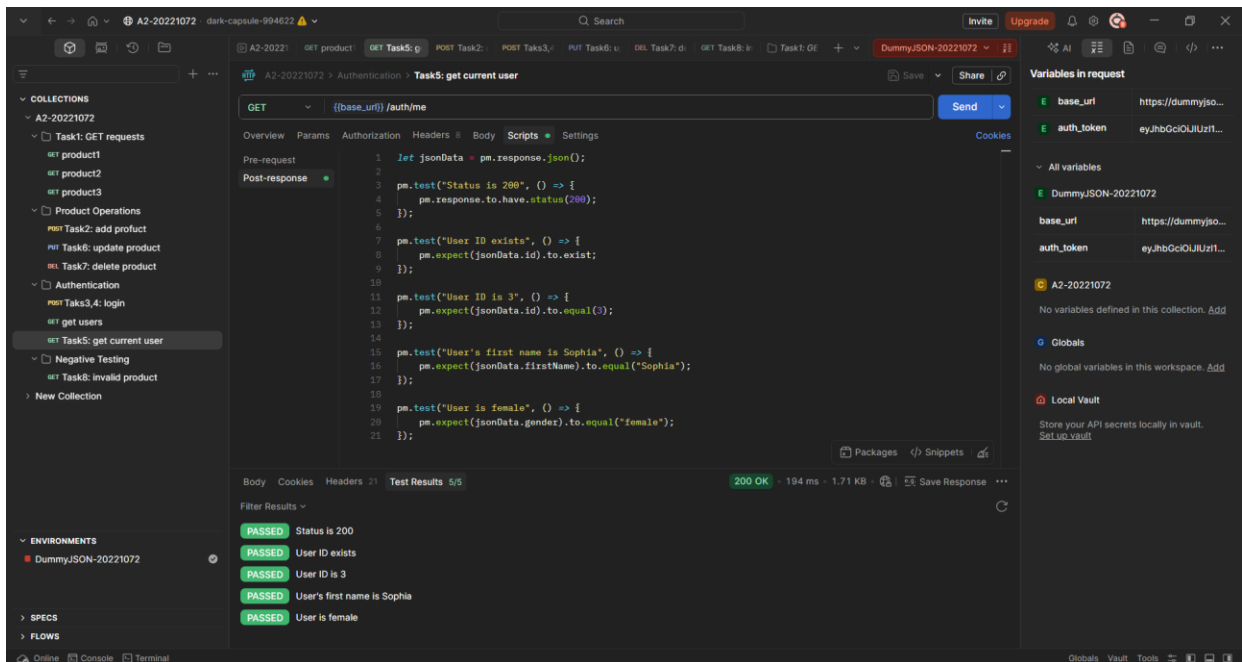
Removed token from cookies and add it to Auth Header



The screenshot shows the Postman interface for the 'Task5: get current user' request. The request is a GET request to the URL 'https://dummyjson.com/auth/me'. The 'Authorization' header is set to 'Bearer {{auth_token}}'. The 'Cookies' tab is active, showing the 'auth_token' cookie. The 'Body' tab shows a JSON response with user details.

Key	Value	Description
Cache-Control	no-cache	
Postman-Token	<calculated when request is sent>	
Host	<calculated when request is sent>	
User-Agent	PostmanRuntime/7.53.0	
Accept	/*	
Accept-Encoding	gzip, deflate, br	
Connection	keep-alive	
Authorization	Bearer {{auth_token}}	

```
1 {
2   "id": 3,
3   "firstName": "Sophia",
4   "lastName": "Brown",
5   "maidenName": "",
6   "age": 43,
7   "gender": "female",
8   "email": "sophia.brown@dummyjson.com",
9   "phone": "+91 210-692-2785",
10  "username": "sophiab",
11  "password": "sophiabpass",
12  "birthDate": "1982-11-6",
13  "image": "https://dummyjson.com/icon/sophiab/120",
14  "bloodGroup": "O+",
15  "height": 177.72,
16  "weight": 52.6,
17  "eyeColor": "Hazel",
18 }
```



The screenshot shows the Postman interface for the 'Task5: get current user' request. The request is a GET request to the URL 'https://dummyjson.com/auth/me'. The 'Authorization' header is set to 'Bearer {{auth_token}}'. The 'Test Results' tab is active, showing the test results for the request.

```
1 let jsonData = pm.response.json();
2
3 pm.test("Status is 200", () => {
4   pm.response.to.have.status(200);
5 });
6
7 pm.test("User ID exists", () => {
8   pm.expect(jsonData.id).to.exist;
9 });
10
11 pm.test("User ID is 3", () => {
12   pm.expect(jsonData.id).to.equal(3);
13 });
14
15 pm.test("User's first name is Sophia", () => {
16   pm.expect(jsonData.firstName).to.equal("Sophia");
17 });
18
19 pm.test("User is female", () => {
20   pm.expect(jsonData.gender).to.equal("female");
21 });
```

Test Results 5/5

- PASSED Status is 200
- PASSED User ID exists
- PASSED User ID is 3
- PASSED User's first name is Sophia
- PASSED User is female

Figure 5: Authenticated request retrieving current user data

6. PUT Update Product

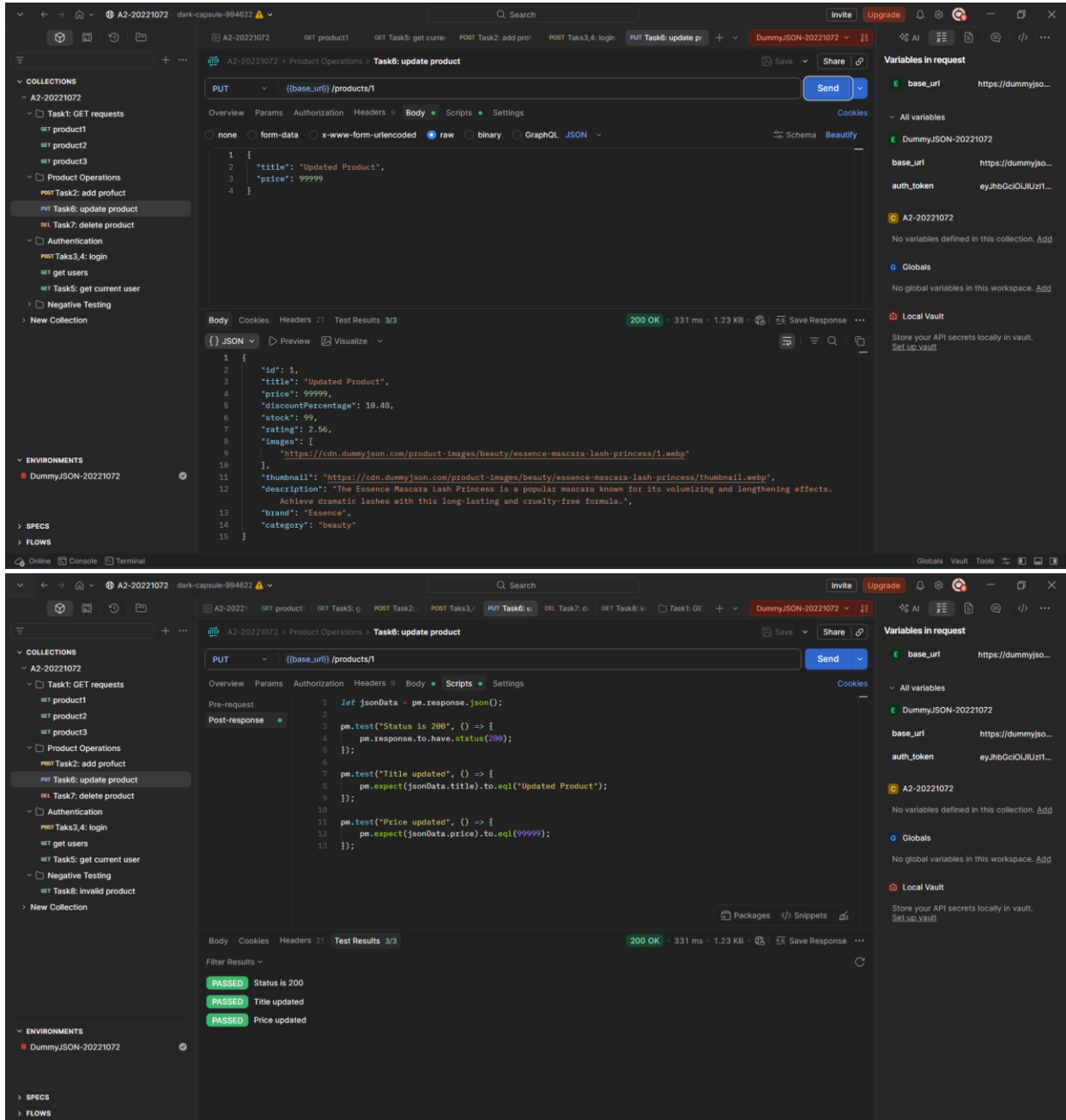


Figure 6: PUT request updating product information

7. DELETE Product

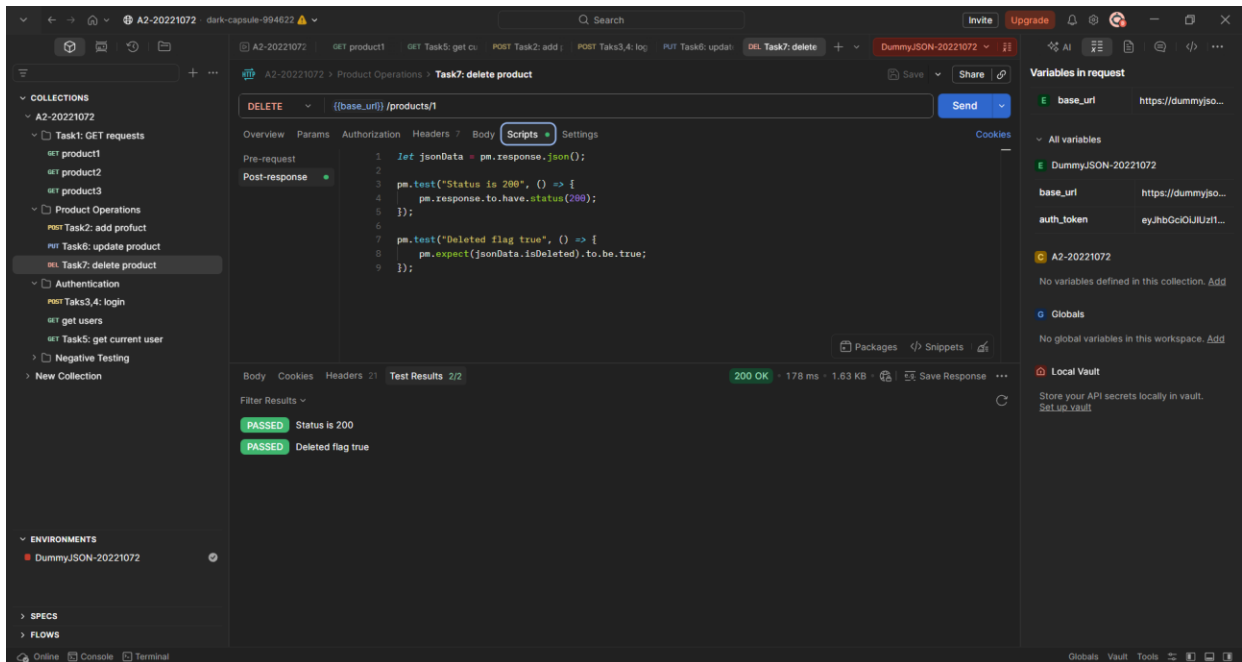
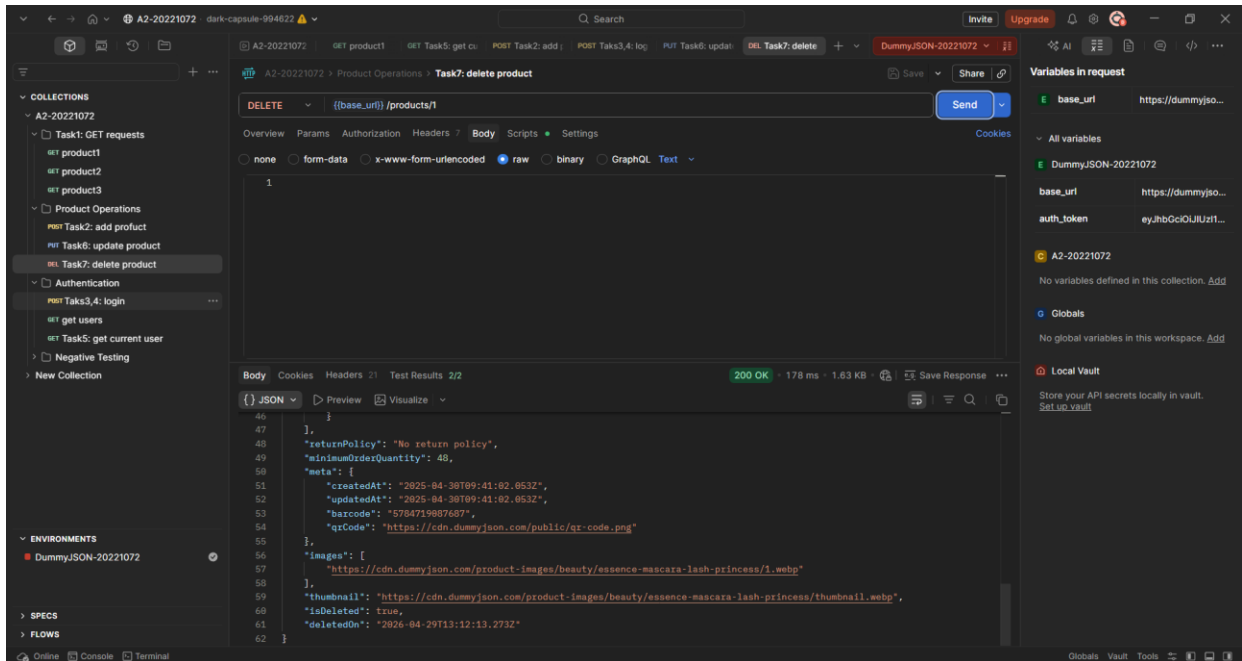


Figure 7: DELETE request confirming product removal

8. Negative Test

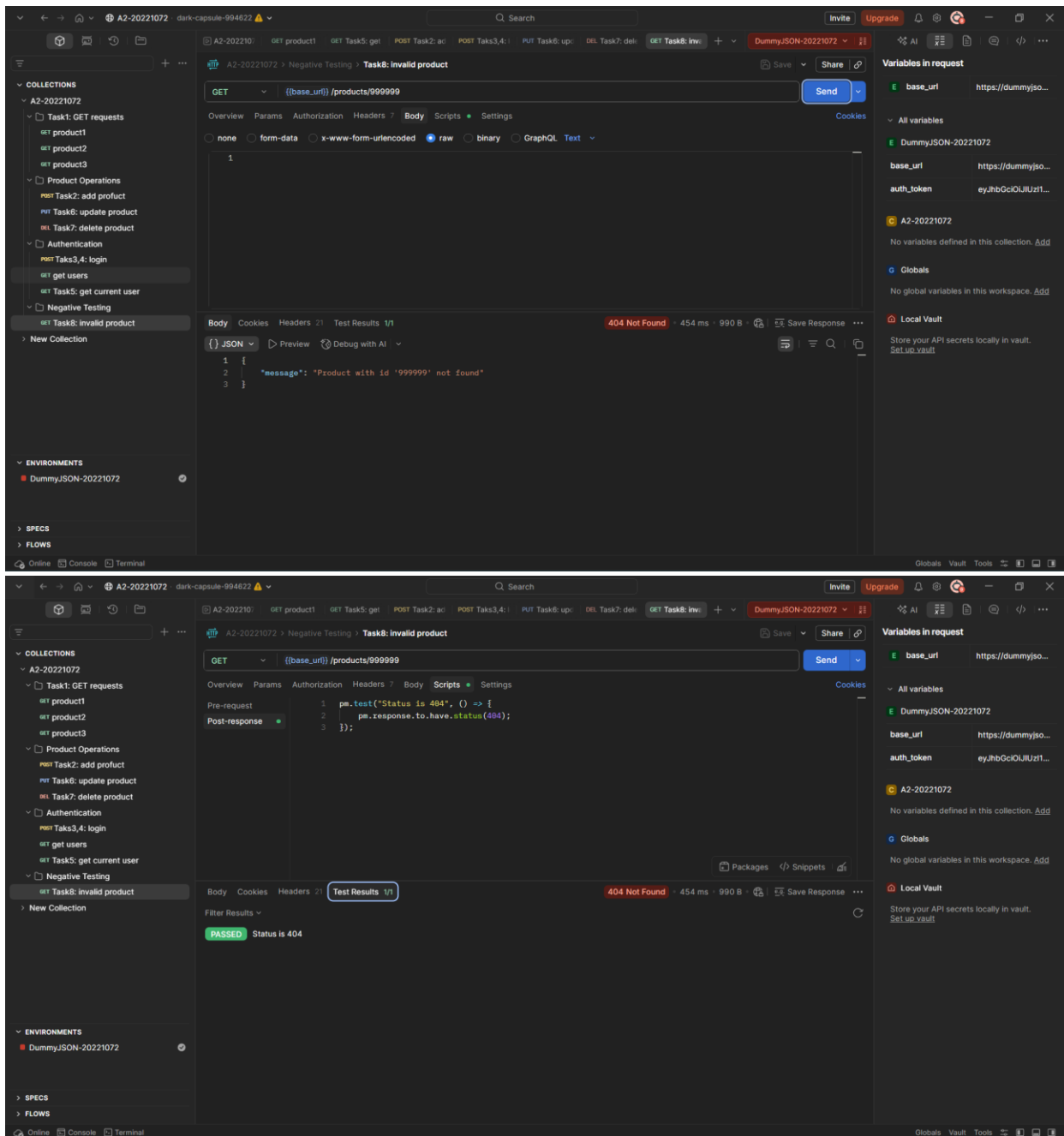


Figure 8: Negative test case validating proper error handling (404 Not Found)

9. Collection Run Results

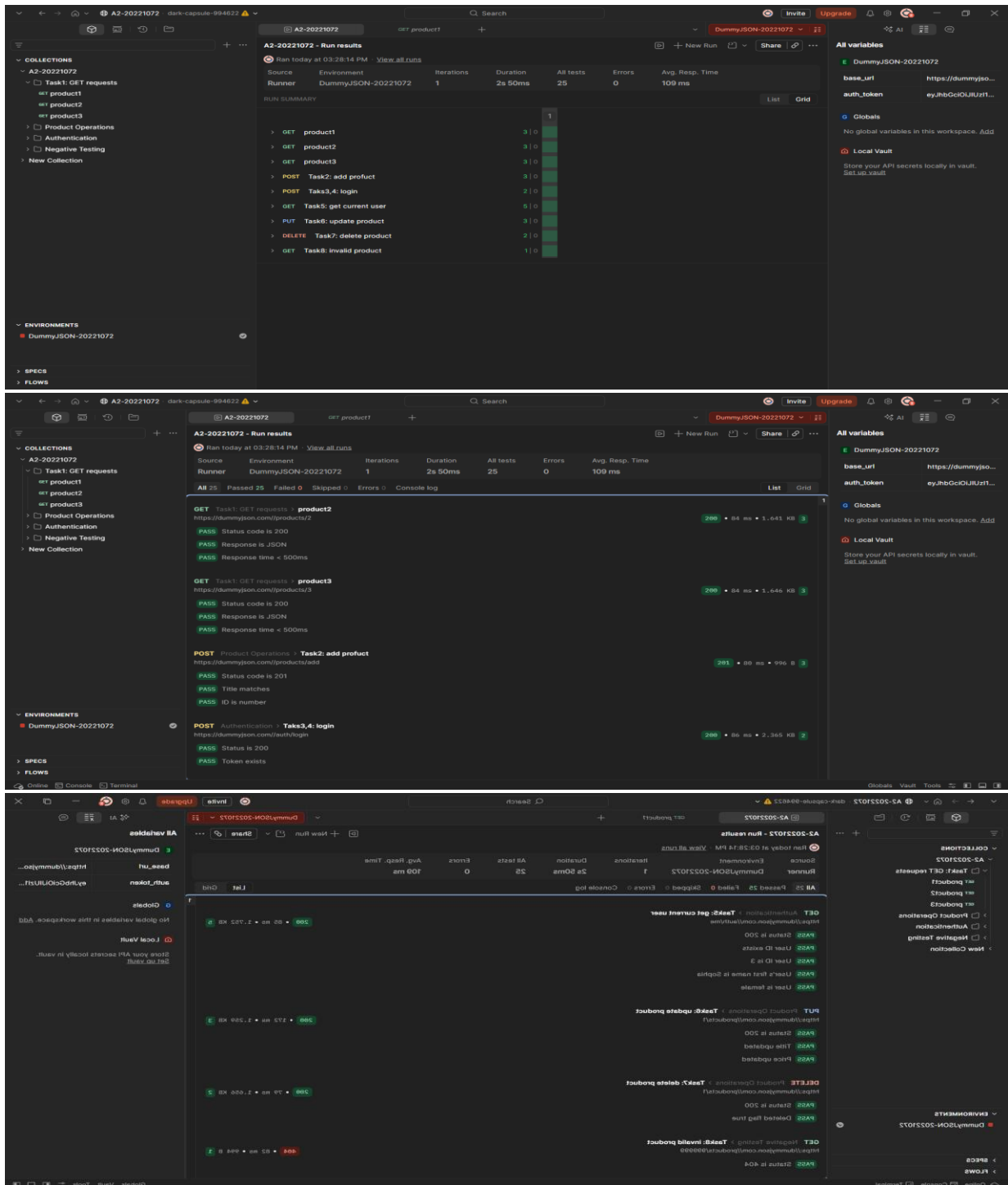


Figure 9: Full collection execution with all tests passing successfully

10. Summary

Testing Approach

This test suite was built using Postman to validate the DummyJSON API by covering CRUD operations and authentication. Automated tests were added to verify status codes, response data, and performance. Environment variables were used to handle the base URL and authentication token dynamically. Negative testing was also included to check error handling. The collection was executed using the Collection Runner to ensure all requests pass successfully in sequence.